

SQL INJECTION



Vettori d’attacco e tecniche difensive

PRESENTED BY

DAVIDE GALANO • MICHELE LORIS ILLUZZI • SIMONE IUNCO • EMANUELE LAPENNA



LA NASCITA DEI SERVIZI WEB DINAMICI

Verso la fine degli anni 90 iniziano ad essere molto più comuni servizi web che prevedono l'accesso a database per la persistenza dei dati degli utenti, e con questi vediamo la comparsa di **form di login** con username e password oppure **tabelle dinamiche** per la gestione di informazioni.

Questa "dinamicità" però può nascondere una vulnerabilità ad uno tra gli attacchi informatici più temibili: l'**SQL injection**.

Discussioni su questo genere di attacco sono iniziate su riviste indipendenti di divulgazione tecnica, come **Phrack Magazine**, che nel 1998 pubblica un articolo denominato "**NT Web Technology Vulnerabilities**" dove l'attacco viene descritto per la prima volta online, facendo uso del web server IIS di casa Microsoft.



ATTACCHI CHE HANNO FATTO LA STORIA DELLA SQL INJECTION



Heartland Payment Systems - 2008

Compromesse **130 milioni** di carte di credito/debito, è considerato uno dei più grandi furti di dati finanziari della storia.

L'attacco è avvenuto utilizzando l'SQLi come punto d'accesso per iniettare uno sniffer nella rete della Heartland.



Sony Playstation Network - 2011

Rubati **1 milione** di account Sony Pictures e **77 milioni** di account PSN.

Un gruppo di hacker noto come i LulzSec prese di mira Sony trovando **vulnerabilità SQLi elementari** nei form di ricerca dei siti di Sony. Gli hacker dichiararono pubblicamente che l'iniezione era banale e che le query erano facilmente manipolabili.

Le password degli utenti erano memorizzate in chiaro nei server Sony!



TalkTalk - 2015

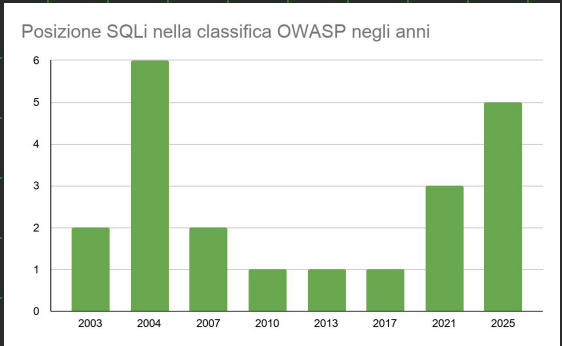
Esfiltrati dati sensibili (inclusi dettagli bancari) di oltre **150.000** clienti.

A mettere in ginocchio TalkTalk fu un gruppo di ragazzini, tra cui un quindicenne, che ha semplicemente utilizzato software di automazione gratuiti scaricabili online, indirizzati verso il sito della multinazionale.

TalkTalk registrò perdite per circa **42 milioni di sterline**.

GRAVITÀ DELL'ATTACCO AL GIORNO D'OGGI

Come abbiamo visto, anche colossi come Sony possono commettere errori così banali, è proprio per questo che la minaccia posta da questo genere di attacchi continua ad essere un argomento attuale. Tutt'ora l'SQL-i rappresenta una delle minacce più imponenti per i sistemi web moderni, posizionandosi al quinto posto nella classifica OWASP del 2025.



OWASP

Open Worldwide Application Security Project: è una fondazione no-profit che si pone l'obiettivo di migliorare la sicurezza informatica nel mondo offrendo risorse di formazione gratuite e supportando lo sviluppo di progetti software.

COS'È L'SQL INJECTION?



L'SQL injection è una tecnica di iniezione di codice utilizzata per attaccare le applicazioni basate sui dati, in cui le istruzioni SQL dannose vengono inserite in un campo di immissione per l'esecuzione.

SQLi DA UN PUNTO DI VISTA TEORICO

Da un punto di vista teorico , l'SQL injection è un problema di **segnalazione in banda** (in-band signaling). Questo fenomeno si verifica quando i dati e i segnali di controllo vengono trasmessi sullo **stesso canale** di comunicazione **senza una distinzione netta**.

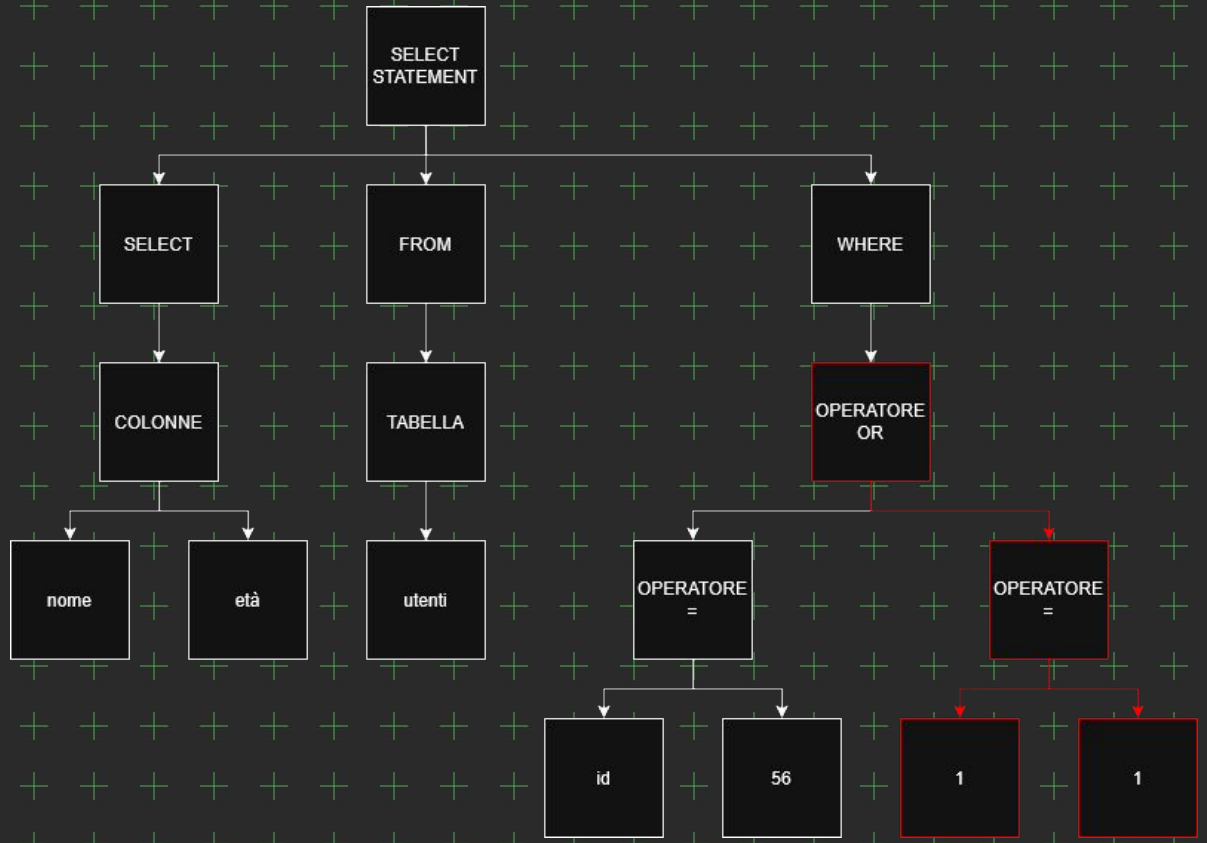
Quando il database riceve una query , utilizza un componente chiamato **parser** per scomporla e costruire un **Abstract Syntax Tree (AST)**, ovvero una struttura ad albero che rappresenta la logica sintattica del metodo .

In una query vulnerabile , l'input malevolo non viene incapsulato come un semplice valore letterale, ma viene interpretato dal parser come **parte integrante della logica**.

L'attaccante riesce così a iniettare nuovi nodi operazionali (come nodi logici `**OR**` o operatori `**UNION**`) che alterano irrimediabilmente la struttura dell'albero sintattico originale, trasformando un dato in un comando esecutivo.



ESEMPIO CONCETTUALE DI AST “AVVELENATO”



L'ANATOMIA E LA MECCANICA DELL'ATTACCO

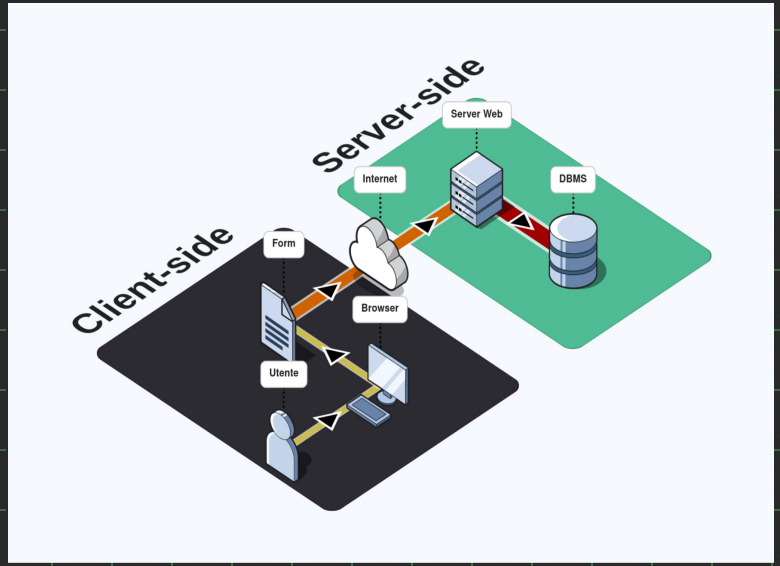
Un attacco **SQL injection** si verifica quando gli aggressori inviano codice eseguibile in form dove l'applicazione si aspetterebbe dati ordinari. Se l'input non è convalidato, il DBMS esegue il codice estraneo.

Fasi principali:

- Identificazione dei vettori (form, parametri GET).
- Costruzione del **payload**.

L'attaccante usa una **virgoletta singola** per terminare la stringa, accoda un comando tramite **punto e virgola (piggybacking)** e aggiunge commenti per ignorare il resto della query legittima.

Se il codice è sintatticamente corretto, bypassa i controlli di programmazione.



TIPOLOGIE DI SQLi

La tecnica dell'iniezione non è universale, ma si declina in diverse metodologie a seconda del bersaglio. Possiamo identificare tre categorie e varianti principali:

IN-BAND (Classic)

La forma tradizionale dove i risultati sono visibili direttamente.

- **Error-Based:** Forza errori per leggere dati nei messaggi di sistema.
- **Union-Based:** Sfrutta l'operatore UNION per estrarre tabelle.

INFERENTIAL (Blind)

L'app non restituisce dati palesi. Si basa su risposte binarie.

- **Boolean-Based:** Verifica se la pagina cambia tra VERO e FALSO.
- **Time-Based:** Usa comandi come *SLEEP* per confermare vulnerabilità tramite ritardi.

OUT-OF-BAND (OOB)

Utilizzata quando i canali diretti sono protetti.

- Il database effettua richieste esterne (DNS/HTTP) per inviare i dati rubati all'attaccante.
- Richiede funzionalità specifiche del DBMS attive.

VARIANTI SPECIFICHE E COMBINATE

DBMS-Specific: Attacchi cuciti su misura per sfruttare sintassi esclusive (es. MS-SQL vs MySQL).

Compounded SQLi: Combinata con altri vettori come XSS, DDoS o DNS hijacking per massimizzare il danno.



ATTACCO SQLi BASATO SU TAUTOLOGIA

Una tautologia è un'affermazione sempre vera. Nei payload SQLi si usa per forzare il database a ignorare le condizioni di sicurezza, come il controllo della password.

PAYLOAD DELL'ATTACCANTE

```
' OR '1'='1
```

LOGICA DI MANIPOLAZIONE

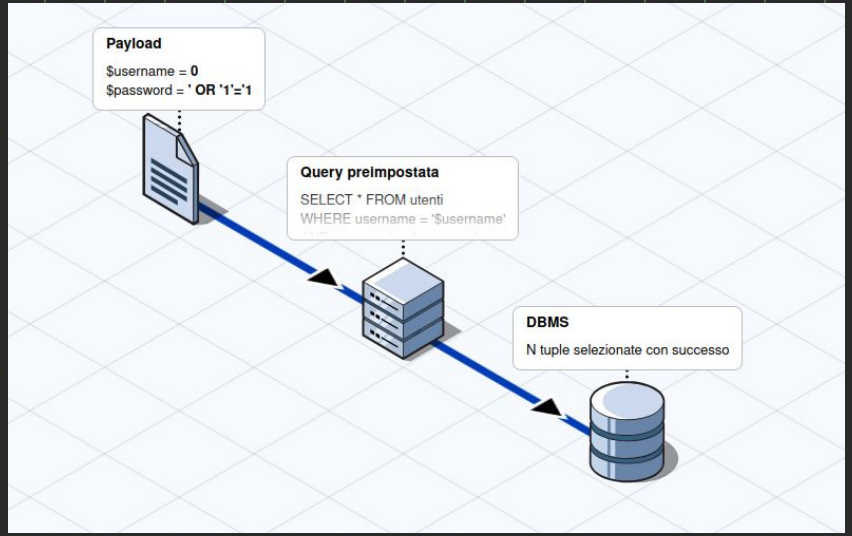
QUERY ORIGINALE:

```
SELECT * FROM utenti
WHERE username = '$username'
AND password = '$password'
```

QUERY MANIPOLATA:

```
SELECT * FROM Utenti
WHERE username = '0'
AND password = '' OR '1'='1'
```

La clausola WHERE diventa sempre VERA. L'attaccante accede come amministratore bypassando la password.



BOOLEAN BASED SQLi

L’espressione ‘1’=’1’ è di fatto un’espressione booleana. In questo scenario l’applicazione non mostra i dati a schermo ma si limita a caricare una pagina diversa se la query ha successo. L’attaccante può ora fare domande mirate al database per esfiltrare dati.

PAYLOAD DELL'ATTACCANTE

```
Bugatti’ AND (SELECT SUBSTRING(password, 1, 1) FROM utenti WHERE username=’admin’)=’A’--
```

LOGICA DI MANIPOLAZIONE

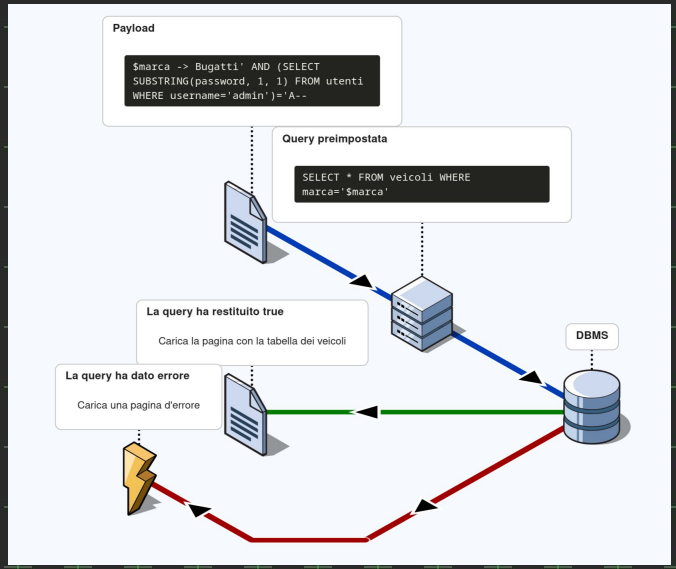
QUERY ORIGINALE:

```
SELECT * FROM veicoli WHERE marca = '$marca'
```

QUERY MANIPOLATA:

```
SELECT * FROM veicoli WHERE marca = 'Bugatti'
AND (SELECT SUBSTRING(password, 1, 1) FROM
utenti WHERE username='admin')='A'--'
```

Andando per tentativi, oppure utilizzando strumenti software possiamo scoprire tutta la password.



ATTACCO SQLi CLASSICO BASATO SULL OPERATORE DI UNIONE

L'obiettivo dell'attacco è di realizzare, partendo da una tabella nascosta, una tabella coerente a quella mostrata a schermo, per poi unirla ad una mostrata dalla pagina sfruttando l'operatore insiemistico UNION, che combina i risultati delle due query separate in un'unica tabella finale.

PRODOTTI

ID	Nome Generico	Descrizione	Prezzo	Quantità	Tipologia
1	Pane	Pagnotta di grano duro appena sfornata	1.5	11	Alimentari
2	Pasta	Bucatini	0.75	86	Alimentari
3	Uova	Uova grandi allevate a terra	1.6	54	Alimentari
4	Biscotti	Gocciole	2.3	72	Alimentari
5	Piatti di plastica	Piatti di plastica riutilizzabili	3.5	64	Casalinghi
6	Acqua	Acqua minerale naturale	0.2	51	Alimentari
7	Batterie AA	Batterie AA a lunga durata	4	19	Elettronica

U

UTENTI

ID	Username	Password	Email	Telefono	Admin
0	wheel	6e6bc4e49dd477ebc98ef4046c067b5f	NULL	NULL	TRUE
1	brodino	043a042c432ddeacccac965626987cb1	brodinocaldo05@gmail.com	3475559876	FALSE
2	ilovetifalockart	26d2e1dc23e6373c45a686d9d66a6f0f	ffvii.tifa.fan99@gmail.com	3204005060	FALSE
3	gojo	81fde0406e833f809140c5eebf552bc0	satorulimitless6@gmail.com	NULL	TRUE
4	brawlpro_xx	f6f4f297487fbe5976d86a9de5dce42a	brawlpro.business.xx@yahoo.com	3921112233	FALSE
5	onepieceisreal	14e230fa08067f5757fab09808ac1b81	nakama.piratecrew@protonmail.com	025550199	FALSE
6	tenz	a8f625ced9d4309f8c0cc4473ff72605	tenz.aim.reflex@icloud.com	0815554321	FALSE

PRODOTTI

ID	Nome Generico	Descrizione	Prezzo	Quantità	Tipologia
1	Pane	Pagnotta di grano duro appena sfornata	1.5	11	Alimentari
2	Pasta	Bucatini	0.75	86	Alimentari
3	Uova	Uova grandi allevate a terra	1.6	54	Alimentari
4	Biscotti	Gocciole	2.3	72	Alimentari
5	Piatti di plastica	Piatti di plastica riutilizzabili	3.5	64	Casalinghi
6	Acqua	Acqua minerale naturale	0.2	51	Alimentari
7	Batterie AA	Batterie AA a lunga durata	4	19	Elettronica
0	wheel	6e6bc4e49dd477ebc98ef4046c067b5f	NULL	NULL	NULL
1	brodino	043a042c432ddeacccac965626987cb1	NULL	NULL	brodinocaldo05@gmail.com
2	ilovetifalockart	26d2e1dc23e6373c45a686d9d66a6f0f	NULL	NULL	ffvii.tifa.fan99@gmail.com
3	gojo	81fde0406e833f809140c5eebf552bc0	NULL	NULL	satorulimitless6@gmail.com
4	brawlpro_xx	f6f4f297487fbe5976d86a9de5dce42a	NULL	NULL	brawlpro.business.xx@yahoo.com
5	onepieceisreal	14e230fa08067f5757fab09808ac1b81	NULL	NULL	nakama.piratecrew@protonmail.com
6	tenz	a8f625ced9d4309f8c0cc4473ff72605	NULL	NULL	tenz.aim.reflex@icloud.com

SECOND ORDER SQLi

Mentre in un attacco SQLi tradizionale il codice malevolo viene eseguito immediatamente non appena l'attaccante invia la richiesta HTTP, in un attacco di secondo ordine il payload malevolo viene prima memorizzato dall'applicazione e solo successivamente eseguito in un contesto diverso.

Fase 1: Iniezione e Archiviazione

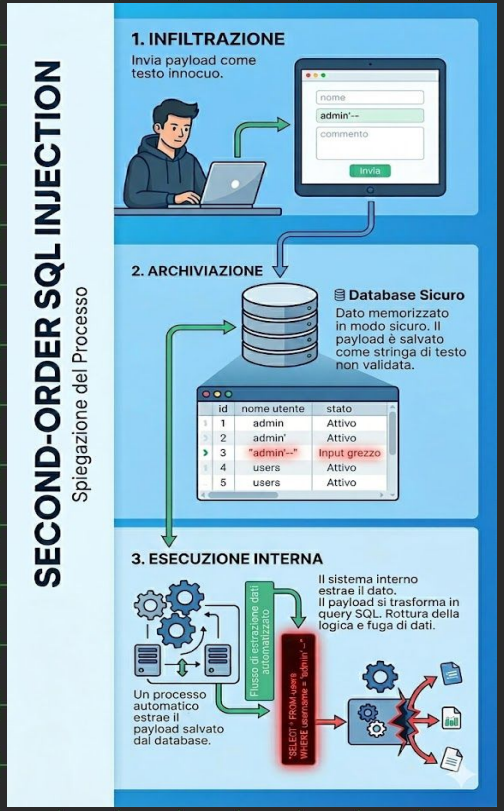
L'attaccante invia un input contenente un payload SQL malevolo all'applicazione web. In questa fase:

- L'applicazione riceve il dato e lo elabora in modo sicuro.
- La query originale non viene compromessa.
- Il payload malevolo ora "riposa" tranquillamente all'interno del DB.

Fase 2: Esecuzione (Il "Secondo Ordine")

Il vero attacco si verifica quando l'applicazione richiama il dato precedentemente memorizzato per utilizzarlo all'interno di una nuova query SQL:

- L'applicazione si "fida" dei dati che provengono dal proprio DB.
- L'applicazione non sanifica il dato estratto prima di concatenarlo in questa nuova query.
- Il payload malevolo viene così interpretato come comando SQL dal database, portando a termine l'attacco



DENIAL OF SERVICE (DoS) TRAMITE SQLi

Spesso si pensa alle SQLi solo come a un modo per rubare dati. Tuttavia, un attaccante può iniettare codice per esaurire completamente le risorse del server, mettendo offline il servizio (Denial of Service).

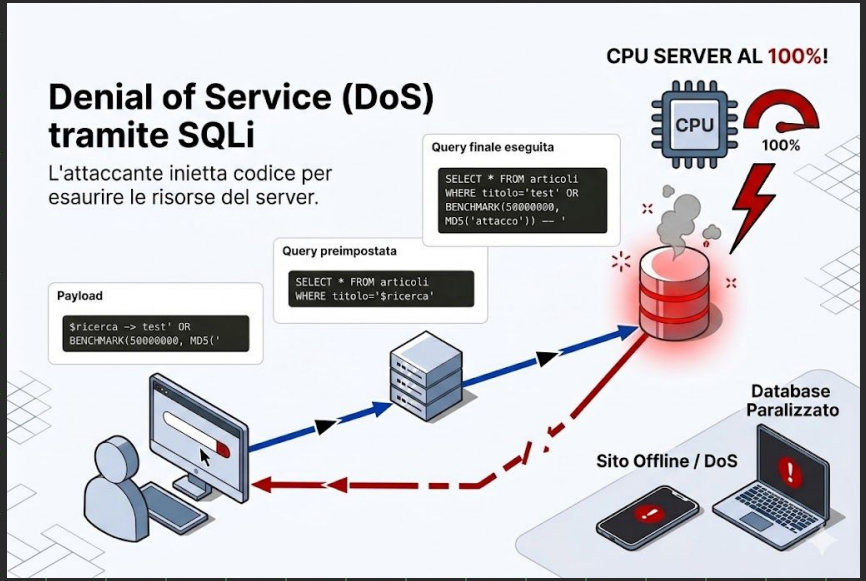
PAYLOAD DELL'ATTACCANTE (esempio MySQL)

```
test' OR BENCHMARK(50000000, MD5('attacco'))--
```

LOGICA DELL'ATTACCO

L'attaccante fa uso della funzione BENCHMARK() che serve originariamente per testare le prestazioni del database. Questo comando usato in modo improprio causa un rallentamento del servizio.

L'effetto:
La CPU del server database schizza istantaneamente al 100%. Se l'attaccante lancia questa query da 10 computer diversi contemporaneamente, il database si paralizza, incapace di rispondere alle richieste legittime degli utenti reali.





SQLi DA UN PUNTO DI VISTA PENALE

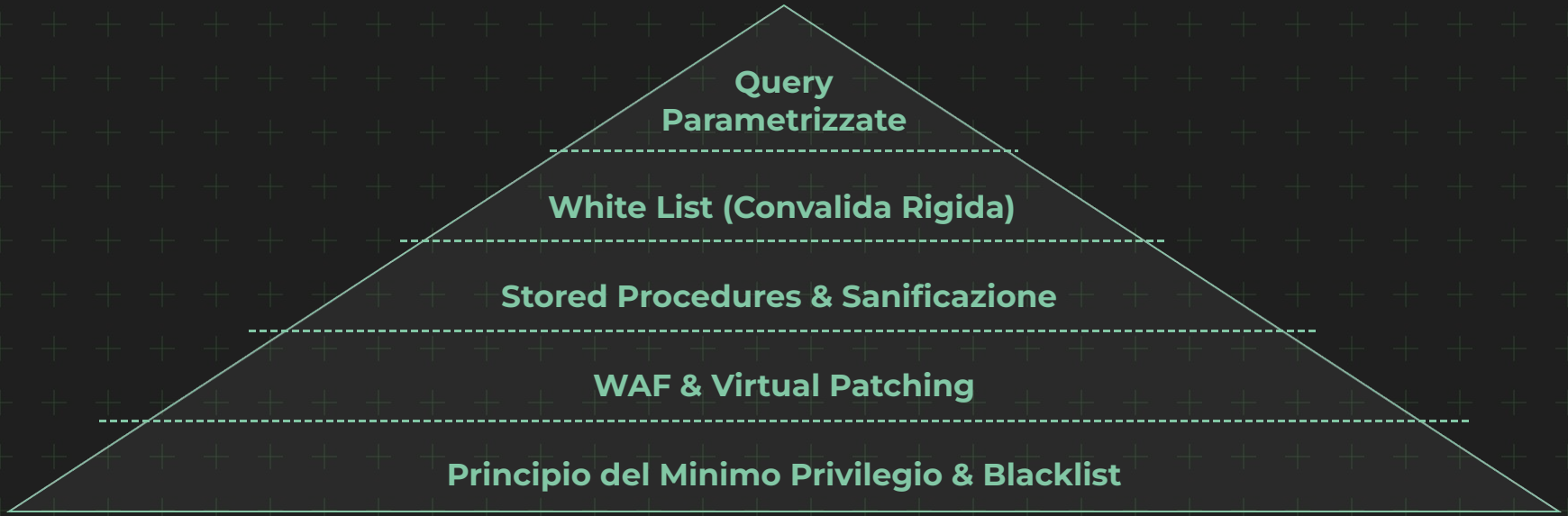
art. 615 ter Codice Penale

“Chiunque abusivamente si introduce in un sistema informatico o telematico protetto da misure di sicurezza ovvero vi si mantiene contro la volontà espressa o tacita di chi ha il diritto di escluderlo, è punito con la reclusione fino a tre anni”

La pena è aumentata (da due fino a dodici anni nei casi più gravi) se il reato è commesso da soggetti con particolari responsabilità o qualifiche, con minaccia o violenza, provoca danni a sistemi o dati, oppure riguarda sistemi di interesse pubblico, militare o legati alla sicurezza, sanità e protezione civile. Nei casi più gravi si procede d’ufficio.

LE METODOLOGIE DI DIFESA

La regola aurea: **mai concatenare input utente non convalidato.**



DIFESA PRIMARIA:
Prepared Statements separano il codice dai dati, rendendo l'iniezione un input inerte.

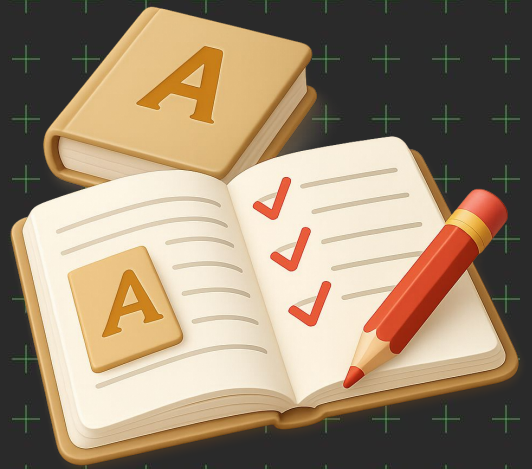
WHITELIST vs BLACKLIST:
Definire il formato accettato (es. solo numeri) è più sicuro che cercare di bloccare pattern infiniti.

DIFESE AGGIUNTIVE:
Il Least Privilege limita i danni post-breach. Il WAF funge da scudo esterno temporaneo.

METTITI ALLA PROVA

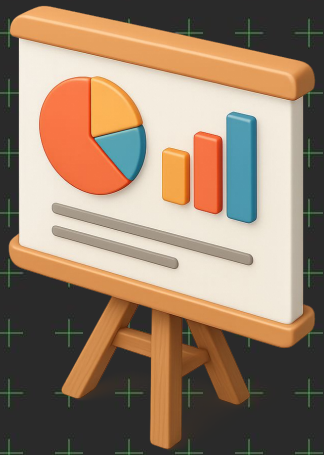
https://forms.gle/KMt5NqDsWyMXSLNH6





DOVE TROVARE QUESTO MATERIALE
&
IL CODICE DELLE DIMOSTRAZIONI PRATICHE

https://github.com/emaxlap-rs/seminario-sql-injection





BIBLIOGRAFIA E SITOGRAFIA UTILIZZATA

1. STANDARD E LINEE GUIDA

OWASP Top 10:2025 - Injection
https://owasp.org/Top10/2025/

NIST SP 800-95: Guide to Secure Web Services
https://csrc.nist.gov/publications/detail/sp/800-95/final

2. DOCUMENTAZIONE E FORMAZIONE

Microsoft Learn: SQL Injection (Transact-SQL)
https://learn.microsoft.com/it-it/sql/relational-databases/security/sql-injection?view=sql-server-ver17

PortSwigger Web Security Academy: SQLi
https://portswigger.net/web-security/sql-injection

3. DIVULGAZIONE E CONTESTO STORICO

Wikipedia: SQL injection
https://it.wikipedia.org/wiki/SQL_injection

Cloudflare Learning Center: SQL Injection
https://www.cloudflare.com/learning/security/threats/sql-injection/

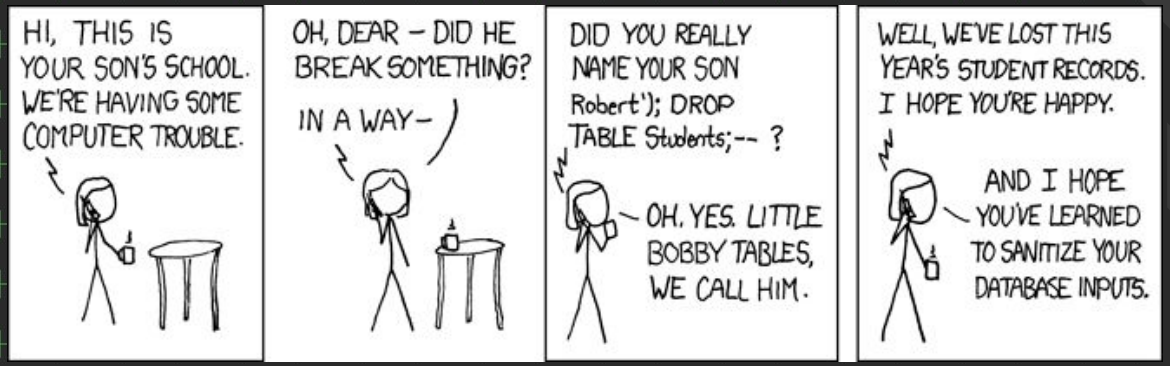
Phrack: Rain Forest Puppy (1998)
https://phrack.org/issues/54/8

4. TESTI E PUBBLICAZIONI ACCADEMICHE

Clarke, J. (2012). SQL Injection Attacks and Defense. Syngress.
https://www.sciencedirect.com/book/9781597494243/sql-injection-attacks-and-defense

Halfond, W. G. (2006). A Classification of SQLi Attacks. IEEE.
https://viterbi-web.usc.edu/~halfond/papers/halfond06issse.pdf

Grazie per l'attenzione



https://xkcd.com/327/

MADE BY

DAVIDE GALANO • MICHELE LORIS ILLUZZI • SIMONE IUNCO • EMANUELE LA[illegible]A

ICONE CONCESSE DA: https://www.thiings.co
IMMAGINI REALIZZATE CON L'USO DI: https://www.icewolfsoft.com/fossflow/ e https://gemini.google.com